

JAVA PROGRAMMING MASTERCLASS

套件 (Package)

「組織程式碼的容器，控制誰能存取什麼」

[← 返回目錄](#)

Outline

- 認識套件 — 什麼是套件、為什麼需要套件
- 建立套件 — `package` 宣告、目錄結構
- 匯入套件 — `import`、萬用字元、靜態匯入
- 常用標準套件 — `java.lang`、`java.util`、`java.io` 等
- 存取控制修飾詞 — `public`、`protected`、預設 (package)、`private`
- Java 模組系統 (JPMS) — JDK 9 的 `module-info.java`
- 課堂練習

認識套件

What Is a Package?

什麼是套件？

- 套件（Package）是一組相關類別與介面的集合，對應到檔案系統的目錄
- 解決兩個核心問題：
 - 命名衝突：不同套件可以有相同名稱的類別（如 `java.util.Date` 與 `java.sql.Date`）
 - 存取控制：套件是 Java 存取控制的邊界單位之一

💡 類比：套件就像資料夾，類別就像檔案。不同資料夾裡可以有同名檔案，不會衝突

套件的命名慣例

層級	說明	範例
最外層	以公司或組織的網域名稱反轉	<code>com.google</code> / <code>org.apache</code>
第二層	專案或產品名稱	<code>com.google.gson</code>
第三層以後	功能模組	<code>com.google.gson.stream</code>
- 全部小寫，不使用底線或大寫 - JDK 標準套件以 <code>java.</code> 或 <code>javax.</code> 開頭		

建立套件

Creating Packages

- **package** 必須是原始碼檔案的第一行有效敘述（前面只能有空行或註解）
- 一個 **.java** 檔只能屬於一個套件

```
1  package com.example.model;
2
3  public class Student {
4      private String name;
5      private int age;
6
7      public Student(String name, int age) {
8          this.name = name;
9          this.age = age;
10     }
11     public String getName() { return name; }
12 }
```


套件與目錄結構的對應

套件名稱對應到實體目錄，每個 `.` 是一層目錄：

```
1  src/  
2  └─ com/  
3      └─ example/  
4          └─ model/  
5              └─ Student.java    ← package com.example.model;  
6          └─ service/  
7              └─ StudentService.java ← package com.example.service;
```

- 套件路徑與類別檔案路徑必須完全對應，否則編譯失敗
- 沒有宣告 `package` 的類別屬於預設套件（`default package`），不建議在正式專案使用

匯入套件

Importing Packages

- **import** 讓你在程式碼中直接使用類別的**短名稱**，不需要每次寫全名
- 寫在 **package** 宣告之後、類別宣告之前

```
1 package com.example.service;
2
3 import com.example.model.Student;    // 匯入單一類別
4 import java.util.ArrayList;
5 import java.util.List;
6
7 public class StudentService {
8     private List<Student> students = new ArrayList<>();
9 }
```

💡 不用 **import** 的情況：使用 **java.lang** 套件的類別（如 **String**、**Math**）— 編譯器自動匯入

萬用字元

`import`

```
1  import java.util.*;    // 匯入 java.util 下所有 public 類別
```

比較項目	單一類別 import	萬用字元 import
寫法	<code>import java.util.ArrayList;</code>	<code>import java.util.*;</code>
效能影響	無差異	無差異（編譯後相同）
命名衝突風險	低	較高（可能引入同名類別）
業界慣例	推薦（可讀性高）	不建議（可讀性低）

靜態匯入

`import static`

讓你直接使用類別的靜態成員，不需要寫類別名稱：

```
1  import static java.lang.Math.PI;
2  import static java.lang.Math.sqrt;
3  import static java.lang.Math.pow;
4
5  public class Circle {
6      double radius;
7      Circle(double r) { radius = r; }
8      double area() {
9          return PI * pow(radius, 2);
10     }
11     double hypotenuse(double a, double b) {
12         return sqrt(pow(a, 2) + pow(b, 2));
13     }
14 }
```


全名存取（不使用 import）

不用 `import`，直接寫全限定名稱（Fully Qualified Name）：

```
1  public class Demo {  
2      public static void main(String[] args) {  
3          java.util.ArrayList<String> list = new java.util.ArrayList<  
();  
4          list.add("炭治郎");  
5  
6          java.util.Date utilDate = new java.util.Date();  
7          java.sql.Date sqlDate = new java.sql.Date(  
8              System.currentTimeMillis());  
9      }  
10 }
```

💡 適用時機：同時使用兩個同名的類別（如 `java.util.Date` 與 `java.sql.Date`），至少其中一個用全名

常用標準套件

Standard Library Packages

JDK 常用標準套件

套件	說明	常用類別
<code>java.lang</code>	核心類別，自動匯入	<code>String</code> 、 <code>Math</code> 、 <code>Integer</code> 、 <code>System</code>
<code>java.util</code>	工具類別與集合框架	<code>ArrayList</code> 、 <code>HashMap</code> 、 <code>Scanner</code> 、 <code>Date</code>
<code>java.io</code>	傳統 I/O	<code>File</code> 、 <code>FileReader</code> 、 <code>BufferedReader</code>
<code>java.nio</code>	新 I/O (非阻塞)	<code>Path</code> 、 <code>Files</code> 、 <code>Channels</code>
<code>java.net</code>	網路	<code>URL</code> 、 <code>Socket</code> 、 <code>URLConnection</code>
<code>java.sql</code>	資料庫 (JDBC)	<code>Connection</code> 、 <code>Statement</code> 、 <code>ResultSet</code>

java.time

套件 (JDK 8)

JDK 8 引入全新的日期時間 API，取代 `java.util.Date`：

類別	說明
<code>LocalDate</code>	日期 (年月日)，不含時間
<code>LocalTime</code>	時間 (時分秒)，不含日期
<code>LocalDateTime</code>	日期 + 時間
<code>ZonedDateTime</code>	帶時區的日期時間
<code>DateTimeFormatter</code>	格式化/解析日期時間字串

```
1 import java.time.LocalDate;
2 import java.time.format.DateTimeFormatter;
3
4 LocalDate today = LocalDate.now();
5 DateTimeFormatter formatter = DateTimeFormatter ofPattern( "yyyy/MM/dd" );
```

存取控制修飾詞

Access Modifiers

四種存取控制修飾詞

修飾詞	同類別	同套件	子類別 (不同套件)	任意類別
private	O	X	X	X
(預設) package	O	O	X	X
protected	O	O	O	X
public	O	O	O	O

- 存取範圍：**private** < 預設 (package) < **protected** < **public**

private

一 最嚴格的限制

- 只能在同一個類別內存取
- 主要用於成員變數（實現封裝）和只供內部使用的方法

```
1  public class BankAccount {  
2      private double balance;  
3  
4      public void deposit(double amount) {  
5          if (amount > 0) balance += amount;  
6      }  
7      public double getBalance() {  
8          return balance;  
9      }  
10     private void log(String msg) {  
11         System.out.println("[LOG] " + msg);  
12     }  
13 }
```


預設 (package-private) – 套件內共享

- 不加任何修飾詞時的預設存取等級
- 同一套件內的所有類別可存取，套件外的類別無法存取

```
1  package com.example.util;
2
3  class Helper {           // 沒有 public，只有同套件能用
4      static String format(String s) {
5          return s.trim().toLowerCase();
6      }
7  }
8
9  public class StringUtils {
10     public static String clean(String input) {
11         return Helper.format(input); // 同套件，可存取
12     }
13 }
```


protected

一 保護繼承鏈

- 同套件的類別可存取
- 不同套件的子類別也可存取 (透過繼承)

```
1 package com.example.animal;  
2  
3 public class Animal {  
4     protected String name;  
5     protected void breathe() {  
6         System.out.println(name + " 在呼吸");  
7     }  
8 }
```

```
1 package com.example.pet;    // 不同套件  
2 import com.example.animal.Animal;  
3  
4 public class Dog extends Animal {  
5     public void show() {  
6         name = "小黑";    // 子類別可存取 protected 成員  
7         breathe();  
8     }  
9 }
```


public

— 完全開放

- 任何套件的任何類別都可存取
- 每個 `.java` 檔案最多只能有一個 `public` 類別，且名稱必須與檔案名稱一致

```
1  package com.example.model;
2
3  public class Student {
4      private String name;
5      private int age;
6
7      public Student(String name, int age) {
8          this.name = name;
9          this.age = age;
10     }
11     public String getName() { return name; }
12     public int getAge() { return age; }
13 }
```

存取控制修飾詞可套用的對象

修飾詞	類別	成員變數	方法	建構方法
private	X (不能用在頂層類別)	O	O	O
預設 (package)	O	O	O	O
protected	X (不能用在頂層類別)	O	O	O
public	O	O	O	O

💡 頂層類別只能是 **public** 或預設 (package-private)，不能是 **private** 或 **protected**

封裝設計原則

原則	說明
成員變數用 private	保護資料，避免外部直接竄改
公開方法用 public	透過方法控制存取，可加入驗證邏輯
繼承用的方法用 protected	子類別能用，但不暴露給外部
內部工具方法用 private	實作細節隱藏在類別內

```
1  public class Temperature {
2      private double celsius;
3
4      public void setCelsius(double c) {
5          if (c < -273.15) throw new IllegalArgumentException("低於絕對零
度");
6          this.celsius = c;
7      }
8      public double getCelsius() { return celsius; }
9      public double getFahrenheit() { return celsius * 9.0/5 + 32; }
```

Java 模組系統 (JPMS)

Java Platform Module System – JDK 9

為什麼需要模組系統？

套件 (Package) 的限制

說明

classpath 地獄

大型專案有幾百個 JAR，版本衝突難以管理

存取控制不夠精確

public 類別對整個 JVM 都可見，無法限制套件間的存取

JDK 本身太龐大

嵌入式裝置也要帶整個 JDK

💡 模組 (**Module**) 是一組套件的集合，明確宣告「我提供什麼」與「我需要什麼」，讓相依關係清晰可見

module-info.java

模組用 `module-info.java` 宣告，放在模組的根目錄：

```
1 // src/module-info.java
2 module com.example.myapp {
3     requires java.sql;
4     requires com.google.gson;
5
6     exports com.example.model;
7     exports com.example.service to com.example.ui;
8 }
```

關鍵字

說明

`module`

宣告模組名稱

`requires`

宣告依賴哪些模組

`exports`

宣告哪些套件對外開放

JDK 模組化

JDK 本身從 JDK 9 起已模組化：

```
1  java.base      ← 所有模組的基礎，自動包含 (java.lang、java.util 等)
2  java.sql       ← JDBC 資料庫
3  java.xml       ← XML 處理
4  java.desktop  ← AWT/Swing GUI
5  java.net.http  ← HTTP Client (JDK 11)
```

```
1  // 查看目前 JDK 所有模組
2  java --list-modules
3
4  // 查看特定模組的套件內容
5  java --describe-module java.base
```

課堂練習

Practice

練習：設計套件結構與存取控制

任務說明

設計一個簡單的學生管理系統，要求：

1. 建立套件 `com.school.model`，內有 `Student` 類別：
 - 成員變數 `name` (`private`)、`grade` (`protected`)、`studentId` (`public`)
 - 提供 `public` 的建構方法和 getter/setter
2. 建立套件 `com.school.service`，內有 `StudentService` 類別：
 - `import` 並使用 `Student`
 - 提供 `public` 方法 `printInfo(Student s)` 列印學生資訊
3. 建立 `com.school.Main`，建立 `Student` 物件並呼叫 `StudentService`

練習：解題提示

```
1  // com/school/model/Student.java
2  package com.school.model;
3
4  public class Student {
5      private String name;
6      protected int grade;
7      public String studentId;
8
9      public Student(String studentId, String name, int grade) {
10         this.studentId = studentId;
11         this.name = name;
12         this.grade = grade;
13     }
14     public String getName() { return name; }
15     public void setName(String name) { this.name = name; }
16     public int getGrade() { return grade; }
17 }
```


練習：解題提示（續）

```
1  // com/school/service/StudentService.java
2  package com.school.service;
3
4  import com.school.model.Student;
5
6  public class StudentService {
7      public void printInfo(Student s) {
8          System.out.println("學號：" + s.studentId);
9          System.out.println("姓名：" + s.getName());
10         System.out.println("年級：" + s.getGrade());
11     }
12 }
```

```
1  // com/school/Main.java
2  package com.school;
3
4  import com.school.model.Student;
5  import com.school.service.StudentService;
6
7  public class Main {
8      public static void main(String[] args) {
9          Student s = new Student("S001", "炭治郎", 2);
10         new StudentService().printInfo(s);
11     }
12 }
```

Q & A

課程結束

套件組織程式碼，存取控制保護封裝邊界